

TakasCo - Proje Mimari ve Geliştirme Dokümantasyonu

Hazırlanan Kitle: Proje Değerlendirme Jürisi **Proje Adı:** TakasCo (Mobil Takas ve Paylaşım Platformu)

Mimari: Full-Stack (React Native / Expo & Laravel)

1. Proje Özeti ve Temel Amacı

Amaç: İnsanların artık kullanmadıkları ancak hala değer taşıyan eşyalarını, ihtiyaç duydukları başka eşyalarla takas edebilmelerini sağlayan dijital bir platform yaratmak. Proje, tüketim çılgınlığını azaltarak "Döngüsel Ekonomi" modeline katkı sağlamayı ve israfı önlemeyi hedeflemektedir. **Sonuç:** Kullanıcıların eşyalarını vitrine çıkarabildiği, konum tabanlı filtrelemeler yapabildiği, diğer kullanıcıların eşyalarına takas teklifi gönderebildiği ve bu süreçleri gerçek zamanlı yönetebildiği uçtan uca çalışan bir mobil uygulama (TakasCo) başarıyla geliştirilmiştir.

2. Mimari Yapı ve Teknoloji Yığını (Tech Stack)

Amaç: Projenin binlerce kullanıcıya aynı anda hizmet verebilecek ölçeklenebilir, güvenli ve modern bir yapıya sahip olması.

Sonuç (Kullanılan Teknolojiler):

- Backend (Sunucu Tarafı):** **Laravel 11** kullanılarak RESTful API geliştirildi. API mimarisi tercih edilerek, uygulamanın gelecekte bir web platformuna veya farklı istemcilere kolayca entegre olabilmesi sağlandı.
- Frontend (İstemci Tarafı):** **React Native** ve **Expo Router** ile çapraz platform (iOS & Android) destekleyen mobil uygulama geliştirildi. Modern bir state (durum) yönetimi ve dosya tabanlı yönlendirme (file-based routing) kullanıldı.
- Veritabanı:** **MySQL**, ilişkisel veri tutarlılığı (kullanıcılar, ürünler, takaslar ve favoriler arası ilişkiler) için tercih edildi.
- Güvenlik:** API isteklerinin güvenliği **Laravel Sanctum** kullanılarak JWT/Token tabanlı oturum yönetimi ile sağlandı.

3. Temel Modüller, Kod Analizi ve Amaç-Sonuç İlişkileri

Jürinin projenin teknik derinliğini anlaması adına, geliştirilen ana modüller ve arkasındaki mühendislik kararları aşağıda detaylandırılmıştır.

A. Kimlik Doğrulama ve Güvenlik (Authentication & Security)

- Amaç:** Kullanıcı verilerinin gizliliğini korumak ve sisteme yetkisiz erişimleri engellemek. Mobil tarafta ise token tabanlı bir yapıyla, uygulamanın her kapatılıp açıldığında kullanıcıdan tekrar şifre istememesini (Persistent Login) sağlamak.
- Kod Analizi (Mobil - Axios Interceptor):** Uygulama açılışında, daha önce kaydedilmiş olan yetki belirteci (token) güvenli bellekten çekilerek her isteğe otomatik dahil edilir.

```
// utils/api.ts (Mobil)
import * as SecureStore from 'expo-secure-store';

api.interceptors.request.use(async (config) => {
  // 1. Cihazın şifreli belleğinden kullanıcının token'ını çekiyoruz.
```

```
const token = await SecureStore.getItemAsync('userToken');
if (token) {
    // 2. Eğer token varsa, her HTTP isteğinin "Authorization" başlığına ekliyoruz.
    // Bu sayede backend, isteği kimin yaptığını güvenli bir şekilde tanıyabiliyor.
    config.headers.Authorization = `Bearer ${token}`;
}
return config;
});
```

- **Ne İşe Yarıyor?** Kullanıcı uygulamaya bir kere giriş yaptığında sunucudan dönen özel anahtar (token), cihazın kırılması zor bir bölümünde (SecureStore) saklanır. Axios interceptor adı verilen bu yapı ise, uygulamadan sunucuya giden *her* isteği (örneğin ana sayfayı yenilemek, ürün eklemek) havada yakalar ve içine bu anahtarı yerleştirir. Böylece kullanıcının her sayfada şifre girmesine gerek kalmaz.
- **Sonuç:** Kesintisiz ve son derece güvenli bir kullanıcı oturum yönetimi kuruldu. Daha önceki geliştirme evrelerinde karşılaşılan "401 Unauthorized" hataları bu mimari sayesinde sistemden tamamen temizlendi.

B. Ürün Yönetimi ve Görsel İşleme (Product & Image Handling)

- **Amaç:** Kullanıcının, takas etmek istediği ürünün detaylarını ve resimlerini hızlıca sisteme yükleyebilmesi. Resimlerin sunucuyu yormaması için kırılarak optimize edilmesi.
- **Kod Analizi (Backend - Intervention Image Kullanımı):** ProductController.php içinde yer alan store metodunda gelen fotoğraflar yakalanır ve işlenir.

```
// backend/app/Http/Controllers/Api/ProductController.php

// 1. Intervention Image kütüphanesini başlatıyoruz (GD driver ile).
$manager = new ImageManager(new Driver());

// 2. Mobil uygulamadan gelen orijinal dosyayı RAM'e alıp okuyoruz.
$image = $manager->decode($file->getRealPath());

// 3. Kullanıcı dikey veya yatay çekmiş fark etmez, resmi tam 800x800 kare olacak şekilde kırıyoruz (cover).
$image->cover(800, 800);

// 4. Eğer 'products' klasörü yoksa oluşturup, optimize edilmiş resmi sunucunun diskine yazıyoruz.
Storage::disk('public')->makeDirectory('products');
$image->save(storage_path('app/public/products/' . $fileName));
```

- **Ne İşe Yarıyor?** Kullanıcılar genellikle telefon kamerasıyla çektikleri 5-10 MB'lık devasa ve farklı boyutlardaki (dikdörtgen, dikey) fotoğrafları yüklerler. Eğer bu fotoğraflar olduğu gibi kaydedilirse, hem sunucu diskini hemen dolar hem de mobil uygulamada ürün galerisi yüklenirken internet paketini sömürüp donmalara sebep olur. Bu kod bloğu, gelen resmi alıp standartlaştırır (800x800 kare) ve boyutunu küçültürken optimize eder.
- **Sonuç:** Kullanıcılar yüksek boyutlu fotoğraflar yüklese dahi, sunucu tarafında bu görseller standart hale getirildi ve depolama maliyetlerinden tasarruf sağlandı. Uygulamanın vitrin (Ana Sayfa) açılış

hızı katlanarak arttı.

C. Gelişmiş Takas Mekanizması (Core Trade Engine)

- **Amaç:** Uygulamanın kalbini oluşturan; iki kullanıcının ürünlerini birbirine teklif etmesi ve sürecin veritabanında tutarlı şekilde ilerlemesi.
- **Kod Analizi (Mobil - Dinamik Ana Sayfa Beslemesi):** `index.tsx` sayfasında, kullanıcıya diğer kişilerin onaylanmış takaslık ürünleri listelenir.

```
// mobile/app/(tabs)/index.tsx

const fetchProducts = async () => {
  try {
    // 1. Axios (api) aracılığıyla Laravel sunucumuzdaki '/products' uç
    noktasına GET isteği atıyoruz.
    const response = await api.get('/products');

    // 2. Sunucudan dönen ürünler listesini (JSON) uygulamanın State'ine
    (belleğine) kaydediyoruz.
    setProducts(response.data);
  } catch (error) {
    console.error('Ürünler yüklenirken hata:', error);
  } finally {
    // 3. İşlem başarılı da olsa başarısız da olsa yüklenme animasyonunu
    (Spinner) durduruyoruz.
    setLoading(false);
  }
};
```

- **Ne İşe Yarıyor?** Mobil uygulamanın sunucu ile konuştuğu temel damarlardan biridir. Asenkron (sayfayı dondurmayan) bir işlem ile API'den ürün listesi istenir. Cevap gelene kadar ekranda bir yükleniyor işareti döner. Cevap geldiği anda veriler React'in State'ine atanır ve sayfa eşzamanlı olarak vitrini çizer.
- **Kod Analizi (Backend - Takas Güvenlik Kontrolü):** Takas işlemleri sırasında sadece ilgili kullanıcının işlem yapabilmesi sağlanmıştır.

```
// backend/app/Http/Controllers/Api/ProductController.php (destroy metodu)

// GÜVENLİK KONTROLÜ (IDOR - Insecure Direct Object Reference Engeli)
// $product->user_id: Veritabanındaki ürünün gerçek sahibi
// $request->user()->id: İstek atan kişinin token'dan çözümlenen kimliği
if ($product->user_id != $request->user()->id) {
  return response()->json(['message' => 'Bu işlemi yapma yetkiniz yok.'],
  403);
}
```

- **Ne İşe Yarıyor?** Kötü niyetli bir kullanıcı, başka birisinin ürününün ID'sini tahmin edip API'ye "Sil" veya "Güncelle" isteği yollayabilir. Bu kod bloğu, isteği yapan kişinin yetkisi olup olmadığını denetler. Ürün sahibiyle işlem yapmak isteyen kişi eşleşmezse, işlem veritabanına inmeden sunucu tarafından 403 (Yasak) hatasıyla reddedilir.

- **Sonuç:** Güvenli ve şeffaf bir el sıkışma (Handshake) mimarisi kuruldu. Sadece yetkili kullanıcılar kendi ürünleri üzerinde işlem yapabilir duruma geldi (Data Inconsistency / Veri Tutarsızlığının önlenmesi sağlandı).

D. Konum Bazlı Yapı ve Kullanıcı Arayüzü (UI/UX)

- **Amaç:** Akıcı ve modern bir kullanıcı deneyimi sunmak, kullanıcının ürünü daha detaylı incelemesine (tam ekran resim, swipe özelliği) olanak tanımak.
- **Kod Analizi (Mobil - Ürün Detay Yönlendirmesi):** Ana sayfada listelenen bir ürüne tıklandığında, dinamik bir şekilde ürün detay ekranına yönlendirme yapılır.

```
// mobile/app/(tabs)/index.tsx

// Expo Router (router.push) ile tıklanan ürünün ID'sini dinamik olarak sayfa
yoluna gömüyoruz.
<TouchableOpacity
  style={styles.productCard}
  onPress={() => router.push(`/product/${item.id}`)}
>
  <Image source={{ uri: item.image_path }} />
</TouchableOpacity>
```

- **Ne İşe Yarıyor?** Mobil uygulamalardaki en kritik işlemlerden biri sayfalar arası veri taşımaktır. Expo Router kullanan bu yapı sayesinde, listelenen yüzlerce ürün arasından hangisine tıklandığı anında yakalanır ve dinamik `/product/[id]` yönlendirmesiyle sayfa değiştirilir. İlgili detay sayfası açıldığında bu ID URL'den okunur ve sadece o ürüne ait bilgiler sunucudan çekilir.
- **Sonuç:** Modern mobil uygulama standartlarında pürüzsüz sayfa geçişleri, pan/zoom yapılabilen galeri deneyimi kodlandı. Tıklanan ürünü kaybetmeden anında odaklanılabilmesi sağlandı.

4. Karşılaşılan Temel Zorluklar ve Çözümler

1. **Zorluk:** *Intervention Image Kütüphanesi Sürüm Uyuşmazlığı ve Image Upload Hataları*
 - **Çözüm:** Backend'de eski sürüm kod blokları, Intervention Image v3 standartlarına göre (`ImageManager::read()` yerine `decode()`) refaktör edildi. Form request doğrulama kuralları (Validation Rules) çoklu dosya (Array) yapısına uygun hale getirildi.
2. **Zorluk:** *React Native iOS Simülatöründe Yönlendirme (Redirect) Döngüsü*
 - **Çözüm:** Expo Router'ın `_layout.tsx` dosyasında bulunan korumalı rota (Protected Route) mantığı yeniden yazıldı. AsyncStorage yerine SecureStore kullanılarak State güncellemeleri asenkron hale getirildi.

5. Proje Sonucu ve Gelecek Vizyonu

TakasCo; modern mobil uygulama geliştirme standartlarına (React Native), sağlam bir API altyapısına (Laravel) ve güvenli veritabanı ilişkilerine sahip, teknik anlamda "Production-Ready" (Canlıya Çıkmaya Hazır) olgunlukta bir projedir.

Gelecek Geliştirmeler (Future Work):

- Kullanıcıların takas öncesi haberleşebileceği Socket.io / Reverb tabanlı **Canlı Mesajlaşma (Chat)** entegrasyonu.
- Kargo entegrasyon API'leri ile **Kargo Takip Sistemi**.

6. Ekran Ekran Geliştirme Süreci ve Kod Analizi

Projeyi geliştirirken her ekranın kullanıcıya kusursuz bir deneyim sunması hedeflenmiştir. İşte uygulamanın ana ekranlarındaki kodlama mantığı ve arka planda dönen mühendislik olayları:

6.1. Ana Sayfa (Vitrin) Ekranı (`index.tsx`)

Neler Oluyor? Uygulama açıldığında kullanıcıyı karşılayan ilk ekrandır. Bu ekranda sunucudaki ilanlar yatay kaydırılabilir kategorilerle ve ızgara (grid) şeklinde listelenir. **Kod Mantığı:**

```
const fetchProducts = async () => {
  setLoading(true);
  try {
    const response = await api.get('/products');
    setProducts(response.data);
  } catch (error) {
    console.error('Hata:', error);
  } finally {
    setLoading(false);
  }
};
```

- **Neden Böyle Yazıldı?** Ana sayfanın boş kalmaması için sayfa yüklendiği an (`useEffect` içerisinde) bu fonksiyon tetiklenir. `setLoading(true)` ile ekrana dönen bir ikon konulur, veri çekildikten sonra `finally` bloğu sayesinde hata alınsa bile o ikon kaldırılarak arayüzün donması engellenir. Bu yapı sayesinde UI thread'i (Kullanıcı Arayüzü İş Parçacığı) tıkanmaz ve sayfa akıcı kalır.

6.2. Ürün Ekleme Ekranı (`add.tsx`)

Neler Oluyor? Kullanıcının takas etmek istediği ürünün fotoğraflarını cihaz kütüphanesinden seçtiği ve bilgilerini (başlık, kategori, takas beklentisi) girdiği form ekranıdır. **Kod Mantığı:**

```
const formData = new FormData();
formData.append('title', title);

images.forEach((asset, index) => {
  formData.append('images[]', {
    uri: Platform.OS === 'ios' ? asset.uri.replace('file://', '') : asset.uri,
    name: asset.fileName || `upload_${index}.jpg`,
    type: asset.mimeType || 'image/jpeg',
  } as any);
});
```

- **Neden Böyle Yazıldı?** Dosya (fotoğraf) yükleme işlemleri standart JSON formatıyla API'ye gönderilemez, sunucuya parçalı halde gitmelidir. Bu yüzden `FormData` nesnesi kullanılmıştır. Ayrıca işletim sistemleri (iOS ve Android) dosya yollarını farklı okuduğu için (iOS'ta `file://` öneki olur),

kod içerisinde işletim sistemi kontrolü (Platform.OS) yapılarak bu farklılık normalize edilmiştir. Böylece çapraz platform (cross-platform) yeteneklerinden tam anlamıyla faydalanılmıştır.

6.3. Ürün Detay ve Galeri Ekranı (product/[id].tsx)

Neler Oluyor? Ana sayfadan bir ürüne tıklandığında açılan, ürünün tüm detaylarının, "Takas Beklentisi"nin ve sahibinin görüldüğü ekrandır. Ayrıca fotoğrafa tıklandığında resim tam ekran (Fullscreen Modal) olur. **Kod Mantığı:**

```
const handleScroll = (event: NativeSyntheticEvent<NativeScrollEvent>) => {  
  const slideSize = event.nativeEvent.layoutMeasurement.width;  
  const index = event.nativeEvent.contentOffset.x / slideSize;  
  setCurrentImageIndex(Math.round(index));  
};
```

- **Neden Böyle Yazıldı?** Kullanıcı birden fazla fotoğraf arasında parmağıyla yatay kaydırma (swipe) yaptığında, resmin altındaki noktaların (Pagination Dots) eşzamanlı güncellenmesi gerekir. Bu kod bloğu, kullanıcının ekranı kaydırduğu x eksenindeki piksel değerini (contentOffset.x) cihazın ekran genişliğine (layoutMeasurement.width) bölerek tam olarak kaçınıcı fotoğrafa bakıldığını dinamik olarak hesaplar. Bu matematiksel yaklaşım sayesinde sıfır gecikmeli, premium hissiyatlı bir resim galerisi deneyimi sunulmuştur.

Saygılarımla, jürinin incelemesine sunulur.